

Masqany Enterprise Mobile App - Technical Standards & Architecture Specifications

Document Overview(each module will have different specs,requiremnts and design but implementation must follow these sttandards)

Project: Masqany Mobile App (Expo Enterprise) **Backend Architecture:** Micro-services (different frameworks) **API Base URL:** <https://www.masqany.speqlink.com/api/v1/>
Development Approach: Modular (auth module, user module, etc.)

Table of Contents

1. [State Management Architecture](#)
2. [TanStack Query \(Server State\) Standards](#)
3. [Zustand \(UI State\) Standards](#)
4. [Redux \(Business-Critical Flows\) Standards](#)
5. [Context API Usage Standards](#)
6. [Module Architecture & Folder Structure](#)
7. [API Layer Standards](#)
8. [Performance Optimization Standards](#)
9. [Enterprise Requirements](#)
10. [Deployment & Build Commands](#)

1. State Management Architecture

1.1 Core Principle

No cross-module imports except through shared or services

1.2 Three-Layer State Strategy

State Type	Tool	Responsibility
UI State	Zustand	Modals, loaders, ride status, real-time updates (location, driver movement)
Server State	TanStack Query	Fetching, caching, background refetching, pagination, mutation syncing
Business-Critical Flows	Redux	Payments, bookings, transactions
App Context	Context API (minimal)	Theme, auth session, language

1.3 What Each Tool Handles

TanStack Query Handles:

- Fetching
- Caching
- Background refetching
- Pagination / infinite scroll
- Mutation syncing

TanStack Query Does NOT Handle:

- UI state (modals, tabs, toggles)
- Complex client workflows
- Cross-feature orchestration

2. TanStack Query (Server State) Standards

2.1 Purpose in Masqany

The app has:

- Property listings (high read volume)
- Search/filter engine (dynamic queries)
- Video feed (infinite scroll like TikTok)
- Chat agent (async interactions)
- Booking/payment flows
- Maps (location-based queries)

2.2 Core Capabilities Required

2.2.1 Smart Caching

- Properties don't re-fetch unnecessarily
- Location-based queries cached per coordinate

2.2.2 Background Sync

- Listings refresh without blocking UI
- Chat responses update asynchronously

2.2.3 Mutation Handling

- Register property
- Book property
- Upload documents

2.2.4 Pagination / Infinite Scroll

- Critical for TikTok-style property feed
- Listings pagination

2.3 Query Client Configuration (Mobile-Optimized)

```
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60 * 5,    // 5 minutes
      cacheTime: 1000 * 60 * 30,   // 30 minutes
      retry: 2,
      refetchOnWindowFocus: false, // IMPORTANT for mobile
    }
  }
})
```

```
}  
}  
})
```

Why these values:

- Prevents unnecessary network calls
- Improves battery + performance

2.4 API Layer (Strict Separation)

Rule: No React Query logic here. No hooks here. Pure HTTP only.

```
import axios from 'axios'
```

```
export const apiClient = axios.create({  
  baseURL: 'https://api.masqany.com',  
})
```

```
export const getProperties = (params) => apiClient.get('/properties', { params })
```

2.5 Query Hooks Pattern

```
import { useQuery } from '@tanstack/react-query'  
import { getProperties } from '@services/api/property'
```

```
export const useProperties = (filters) => {  
  return useQuery({  
    queryKey: ['properties', filters],  
    queryFn: () => getProperties(filters),  
  })  
}
```

Why this is powerful:

- Cache is keyed by filters
- Same query reused across screens
- Automatic deduplication

2.6 Infinite Scroll (TikTok-style Feed) Pattern

```
import { useInfiniteQuery } from '@tanstack/react-query'

export const usePropertyFeed = () => {
  return useInfiniteQuery({
    queryKey: ['feed'],
    queryFn: ({ pageParam = 1 }) => apiClient.get('/feed', { params: { page: pageParam } }),
    getNextPageParam: (lastPage) => lastPage.data.nextPage ?? undefined,
  })
}
```

2.7 Mutations Pattern

```
import { useMutation, useQueryClient } from '@tanstack/react-query'

export const useCreateProperty = () => {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: (data) => apiClient.post('/properties', data),
    onSuccess: () => {
      queryClient.invalidateQueries(['properties'])
    },
  })
}
```

2.8 UI Integration Pattern (Correct Flow)

1. User taps property
2. Zustand sets selected ID
3. Screen uses: `const { data, isLoading } = usePropertyDetails(id)`
4. UI never touches API directly

2.9 Query Key Rules

Bad	Good	Why
-----	------	-----

['properties']	['properties', filters]	Enables cache segmentation, prevents stale data collisions
----------------	-------------------------	--

2.10 Co-location Rule

Keep queries inside module: modules/property/api/queries.ts — **Not global dumping**

2.11 Normalization Rule

Transform inside query - do NOT pass raw backend response everywhere:

```
queryFn: async () => {
  const res = await getProperties()
  return res.data.data // normalize here
}
```

2.12 Placeholder Data Pattern

placeholderData: previousData // Improves UX

2.13 What You Should NEVER Do with TanStack Query

Prohibited	Why
✗ Store server data in Zustand	Lose caching, duplicate state
✗ Call APIs directly in components	Breaks caching, deduplication, retries
✗ Use generic query keys like ['data']	Cache collisions, unpredictable UI
✗ Over-invalidate queries with invalidateQueries()	Use specific keys ['properties']
✗ Ignore error states	Always handle isLoading, isError, retry logic
✗ Set staleTime = 0 (default trap)	Causes constant refetching, battery drain

2.14 Mobile-Specific Enhancements

- **Network Awareness:** Use React Native NetInfo to pause queries when offline, resume when online
- **Prefetching:** Use `queryClient.prefetchQuery()` when navigating to next screen

2.15 Final Mental Model for TanStack Query

A smart data layer that sits between your UI and backend, controlling when, how, and why data flows.

Not using it properly = duplicate state, performance issues, messy architecture

Using it correctly = scalable, predictable

3. Zustand (UI State) Standards

3.1 What Zustand Is in This Context

Zustand is a **minimal, subscription-based state container** for UI state:

- Components subscribe to exact slices of state
- Updates trigger only necessary re-renders
- No providers, no reducers, no boilerplate overhead

3.2 Scope: What Counts as "UI State" in Masqany

Included in Zustand:

- Modal visibility (filters, booking, chat)
- Bottom sheets (property details, map overlays)
- Active tabs / navigation helpers
- Loading flags (button-level, not API data)
- Temporary selections (selected property ID)
- Media playback state (video feed controls)

NOT included in Zustand:

- Property data (TanStack Query handles that)

- Auth tokens (hybrid: secure storage + minimal store)
- Business workflows

3.3 Enterprise Pattern: "Slice-based UI Store"

Structure:

```
store/  
  ui/  
    modal.store.ts  
    navigation.store.ts  
    feed.store.ts  
    map.store.ts  
    index.ts
```

Rule: Each file = one concern

3.4 Core Design Principles

3.4.1 Single Responsibility per Store

Don't mix unrelated state.

Bad:

```
{ isModalOpen, selectedProperty, isPlayingVideo, mapRegion }
```

Good:

- modal.store → modal state
- feed.store → video state
- map.store → map state

3.4.2 Flat State (Avoid Deep Nesting)

Bad:

```
ui: { modal: { property: { open: true } } }
```


Good:

isPropertyModalOpen: boolean

Why: Flat state = faster updates + simpler selectors

3.4.3 Actions Co-located with State

```
const useModalStore = create((set) => ({
  isOpen: false,
  open: () => set({ isOpen: true }),
  close: () => set({ isOpen: false }),
}))
```

3.5 Practical Stores for Masqany

3.5.1 Modal Store

```
export const useModalStore = create((set) => ({
  activeModal: null, // 'filter' | 'booking' | 'chat'
  openModal: (name) => set({ activeModal: name }),
  closeModal: () => set({ activeModal: null }),
}))
```

Usage:

```
const activeModal = useModalStore(s => s.activeModal)
const openModal = useModalStore(s => s.openModal)
```

3.5.2 Feed Store (TikTok-style video)

```
export const useFeedStore = create((set) => ({
  activeVideoId: null,
  isMuted: false,
  setActiveVideo: (id) => set({ activeVideoId: id }),
  toggleMute: () => set((s) => ({ isMuted: !s.isMuted })),
}))
```

```
)))
```

Why: Avoids re-rendering the entire feed.

3.5.3 Map Store

```
export const useMapStore = create((set) => ({
  region: null,
  selectedMarkerId: null,
  setRegion: (region) => set({ region }),
  selectMarker: (id) => set({ selectedMarkerId: id }),
}))
```

3.5.4 Navigation Helper Store (optional)

```
export const useNavStore = create((set) => ({
  currentTab: 'home',
  setTab: (tab) => set({ currentTab: tab }),
}))
```

3.6 Performance Optimization (Critical)

3.6.1 Always Use Selectors

Do this:

```
const isOpen = useModalStore(s => s.isOpen)
```

NOT this:

```
const state = useModalStore()
```

Why: Prevents unnecessary re-renders, subscribes only to what you need.

3.6.2 Use Shallow Comparison (for multiple values)

```
import { shallow } from 'zustand/shallow'

const { isOpen, open } = useModalStore(
  (s) => ({ isOpen: s.isOpen, open: s.open }),
  shallow
)
```

3.7 Integration with TanStack Query

Separation:

- **Zustand** → controls UI
- **TanStack Query** → controls data

Example flow:

1. User taps property
2. Zustand: sets selectedPropertyId, opens modal
3. Modal: uses TanStack Query to fetch property details

Why this avoids: storing API data in Zustand, stale duplication.

3.8 Folder-Level Integration Pattern

Inside a feature:

```
modules/property/
  components/
  screens/
  api/      # TanStack Query hooks
  store/    # optional local zustand slice
```

Rule:

- If state is global UI → /store/ui
- If state is feature-specific → module/store

3.9 Middleware (Enterprise Enhancements)

Persist (for UI preferences)

```
import { persist } from 'zustand/middleware'
```

Use for: theme mode, onboarding completion

Devtools

```
import { devtools } from 'zustand/middleware'
```

Use for: development

3.10 Anti-Patterns to Avoid

Anti-Pattern	Why
Turning Zustand into Redux (massive global store, unnecessary abstraction layers)	Violates minimalism
Storing server data in Zustand	TanStack Query handles that
Overusing global state	If state is local → keep it in component

3.11 Minimal Enterprise Blueprint (Setup-Ready)

Create:

- modal.store.ts
- feed.store.ts
- map.store.ts

Add:

- Selectors pattern
- Shallow optimization
- Devtools middleware

Do NOT:

- Implement business logic
- Connect APIs yet

4. Redux (Business-Critical Flows) Standards

4.1 When to Use Redux

Redux is reserved for **business-critical flows**:

- Payments
- Bookings
- Transactions

4.2 Why Redux for These Flows

- Complex state transitions
- Need for predictable state history
- Audit trail requirements
- Multi-step workflows

5. Context API Usage Standards

5.1 Scope (Minimal)

Context API is used **only** for:

- Theme
- Auth session (shell only)
- Language

5.2 Why Not for Other State

- Prop drilling is not scalable
- Context is suitable for low-frequency global state

- For high-frequency updates in large mobile apps, Zustand/Redux are more appropriate (better performance, state isolation)

6. Module Architecture & Folder Structure

6.1 Core Rule

No cross-module imports except through shared or services

6.2 Feature Modularization Rules

- Each feature is self-contained
- No cross-feature imports
- Shared logic only in shared/

6.3 Folder Structure

```
modules/  
  property/  
    components/  
    screens/  
    api/      # TanStack Query hooks  
    store/    # optional local zustand slice  
  auth/  
    components/  
    screens/  
    api/  
  booking/  
    components/  
    screens/  
    api/  
    store/  
  chat/  
    components/  
    screens/  
    api/  
  payments/  
    components/  
    screens/
```

api/
store/ # Redux for business-critical

store/
 ui/
 modal.store.ts
 navigation.store.ts
 feed.store.ts
 map.store.ts
 index.ts

shared/
 components/
 hooks/
 utils/
 constants/
 types/

services/
 api/
 client.ts
 property.ts
 auth.ts
 booking.ts

6.4 Layered Responsibility

- **UI (screens/components)** → render only
- **Query hooks** → data orchestration
- **API layer** → HTTP only
- **TanStack Query** → caching + lifecycle

7. API Layer Standards

7.1 Base Configuration

```
const apiClient = axios.create({  
  baseURL: 'https://api.masqany.com',  
  timeout: 30000,  
  headers: {  
    'Content-Type': 'application/json',  
  },  
})
```

7.2 Environment Configuration

Required environments:

- dev
- staging
- production

Setup:

- .env handling (Expo config)
- API base URL switching
- Feature flags (enable/disable modules remotely)

7.3 Data Contracts with Backend

Must define:

- Strict API response formats
- Pagination structure
- Error schema

Why: Prevents frontend/backend mismatch chaos

8. Performance Optimization Standards

8.1 Selectors (Zustand)

Always use selectors to prevent unnecessary re-renders.

8.2 Shallow Comparison

Use shallow from zustand/shallow for multiple values.

8.3 TanStack Query Mobile Settings

- staleTime: 5 minutes
- cacheTime: 30 minutes
- refetchOnWindowFocus: false

8.4 Performance Budgeting

Define limits early:

Metric	Target
Max screen load time	< 2 seconds
Max bundle size	TBD
Max re-renders per interaction	TBD

Tools:

- React DevTools profiler
- Flipper (React Native debugging)

8.5 Prefetching

Use queryClient.prefetchQuery() when navigating to next screen.

8.6 Network Awareness

Use React Native NetInfo to:

- Pause queries when offline
- Resume when online

9. Enterprise Requirements

9.1 Design System Maturity

Required:

- Type scale (fixed system): e.g., 12 / 14 / 16 / 20 / 24 / 32 (no random sizes)
- Line-height system
- Font weight mapping (don't overuse 500/600 everywhere)
- Consistent spacing scale: e.g., 4, 8, 12, 16, 24, 32

Why it matters: Prevents UI drift across modules, makes Tailwind actually reusable.

9.2 Accessibility (Most Ignored, Critical at Scale)

Key requirements:

- Minimum contrast ratios (gradients may fail this)
- Dynamic font scaling (Android accessibility settings)
- Screen reader support (labels, roles)
- Touch target size ($\geq 44\text{px}$)

Risk if ignored:

- App becomes unusable for a segment of users
- Possible rejection in stores or enterprise deployments

9.3 Navigation Architecture

Requirements:

- Stack vs Tabs vs Modals strategy
- Deep linking support (critical for sharing properties)
- Navigation state persistence
- Lazy loading screens

9.4 Error Handling Strategy (Global)

Required:

- Global error boundary
- API error normalization layer
- Retry strategies (TanStack Query helps here)
- User-friendly fallback UI

Pattern: API → normalized error → UI component decides rendering

9.5 Logging & Observability

Requirements:

- Structured logging (not console.log)
- Error tracking (e.g., Sentry)
- Performance monitoring (slow screens, memory usage)
- API latency tracking

Why: When your app serves millions, debugging locally is useless.

9.6 Offline & Network Resilience (Kenya Context)

Must-have:

- Detect offline state
- Cache last known data (TanStack Query helps)
- Queue mutations (e.g., property uploads)
- Retry when connection restores

9.7 Media Optimization (Critical for Use Case)

App is media-heavy (images, TikTok-style video).

Requirements:

- Image compression pipeline (WebP is good ✓)
- Lazy loading images
- Video streaming strategy (not full downloads)
- CDN-backed delivery

Risk: Without this → app becomes slow, data-heavy, expensive for users

9.8 Security Architecture

Must include:

- Secure token storage (Expo SecureStore)
- API request signing (if needed)
- Role-based access (admin vs user vs owner)
- Input validation (client + server)

Especially for:

- Property ownership verification
- Payments
- Chat agent actions

9.9 CI/CD Pipeline

Use: Expo EAS

Setup: Automated builds

9.10 Internationalization (Important for App)

Supported languages:

- English
- Kiswahili
- Sheng (AI side)

Requirements:

- i18n framework (not hardcoded text)
- Dynamic text rendering
- RTL readiness (future-proofing)

9.11 AI Integration Readiness (Chat Agent)

Prepare:

- Streaming responses (not blocking UI)

- Message state separation (UI vs server)
- Fallback when AI fails

9.12 Testing Strategy

At minimum define:

- Unit tests (stores, hooks)
- Integration tests (API + UI)
- Manual QA flows

9.13 Permissions System (Mobile-Specific)

Required permissions:

- Location (maps)
- Camera (property upload)
- Storage (media)

Plan:

- Permission request flow
- Graceful fallback if denied

9.14 UX Consistency Rules

Define:

- Animation standards
- Transition timing
- Gesture behavior

Why: Consistency = perceived quality

9.15 Documentation (Underrated but Critical)

Document:

- Architecture decisions
- Folder structure rules
- State management rules

- API contracts

9.16 Final Reality Check

Done:

- UI design ✓
- Theme system ✓
- Basic setup ✓

What determines success now: Operational architecture, not visuals

If you ignore these: App will work... then collapse at scale

If you implement them: You'll have a production-grade system from day one

10. Deployment & Build Commands

10.1 Commands to Ensure Deployment Works

(To be computed after every module completion and confirmation)

1. Check Expo doctor for issues

```
npx expo-doctor
```

2. Remove dependencies and native folders

```
rm -rf node_modules
```

```
rm -rf android
```

```
rm -rf .expo
```

3. Clean install

```
pnpm install
```

4. Prebuild with clean flag

```
npx expo prebuild --clean
```

5. Build for Android (preview profile with cache clear)

```
pnpm dlx eas-cli build -p android --profile preview --clear-cache
```

10.2 Module Completion Checklist

- Module is complete
- Awaiting API endpoint only
- Confirmation received
- Deployment commands executed

Appendix: Quick Reference

State Management Decision Tree

What am I managing?	Tool
Modal open/close, loading flags, selected item ID, video mute state	Zustand
Property listings, search results, chat responses, booking records	TanStack Query
Payment flow, booking transaction, multi-step workflows	Redux
Theme, auth session (basic), language	Context API

Prohibited Patterns Summary

Pattern	Why It's Prohibited
Cross-module imports (except shared/services)	Breaks modularity, creates coupling
Storing server data in Zustand	Duplicates state, loses caching
Calling APIs directly in components	Breaks caching, retries, deduplication
Generic query keys ['data']	Cache collisions, unpredictable UI
Over-invalidation of queries	Performance issues
Setting staleTime: 0 on mobile	Constant refetching, battery drain
Deeply nested Zustand state	Slower updates, complex selectors

Turning Zustand into Redux	Violates minimalism principle
----------------------------	-------------------------------

End of Masqany Mobile App Technical Standards Document